

Installare Python su Windows 10/11 con Miniconda, Conda, Windows Terminal e Visual Studio Code

Questa guida descrive una configurazione semplice e ordinata per utilizzare **Python su Windows 10/11** durante il corso, usando **Prompt dei comandi (cmd)** all'interno di **Windows Terminal**.

L'idea è:

```
Miniconda
  ↓
Conda
  ↓
Prompt dei comandi (cmd)
  ↓
Visual Studio Code
  ↓
Python + estensioni Python
```

In questo modo Python e i pacchetti utilizzati nel corso rimangono isolati dal resto del sistema.

Indice

1. [Installare Visual Studio Code](#)
2. [Installare Miniconda](#)
3. [Rendere Conda disponibile nel Prompt dei comandi di Windows Terminal](#)
4. [Creare un ambiente Conda dedicato al corso](#)
5. [Attivare l'ambiente del corso](#)
6. [Isolare l'ambiente con PYTHONNOUSERSITE](#)
7. [Verificare Python](#)
8. [Aggiornare pip](#)
9. [Installare i package con Conda](#)
10. [Interprete Python e kernel Jupyter](#)
11. [Avviare Visual Studio Code dall'ambiente Conda](#)
12. [Installare le estensioni Python e Jupyter in VS Code](#)
13. [Selezionare il kernel in VS Code](#)
14. [Flusso di lavoro consigliato durante il corso](#)
15. [Terminare la sessione](#)
16. [Problemi comuni](#)
17. [In caso di problemi con VS Code / Jupyter](#)
18. [Riepilogo dei comandi](#)
19. [Concetto fondamentale](#)

1. Installare Visual Studio Code

Se Visual Studio Code non è già installato:

1. scaricare **Visual Studio Code** dal sito ufficiale Microsoft;
2. eseguire il programma di installazione;
3. lasciare attive le opzioni standard;
4. verificare che il comando `code` sia disponibile da terminale.

Aprire **Windows Terminal** usando il profilo **Prompt dei comandi** e digitare:

```
code --version
```

Se viene visualizzato il numero di versione di VS Code, il comando `code` è configurato correttamente.

2. Installare Miniconda

Miniconda è una distribuzione minimale che installa:

- Python;
- `conda`, il gestore degli ambienti e dei pacchetti;
- pochi altri componenti essenziali.

È più leggera di Anaconda e permette di installare soltanto ciò che serve.

Scaricare dal sito ufficiale la versione di **Miniconda per Windows 64 bit** ed eseguire il file `.exe`.

E' possibile **aggiornare conda**, in caso di nuova versione, con

```
conda update -n base
```

oppure (se la nuova versione fosse, per ipotesi, la 26.7.2):

```
conda install -n base -c defaults conda=26.7.2
```

`base` è l'ambiente dove vive conda.

Impostazioni consigliate

Durante l'installazione:

- scegliere **Just Me**, salvo esigenze particolari;
 - lasciare la cartella di installazione proposta;
 - **non è necessario aggiungere Miniconda al PATH di Windows;**
 - completare l'installazione con le opzioni predefinite.
-

3. Rendere Conda disponibile nel Prompt dei comandi di Windows Terminal

Dopo l'installazione di Miniconda, può essere necessario inizializzare Conda una sola volta per **cmd.exe**.

Dal menu **Start** aprire:

Miniconda Prompt

e digitare:

```
conda init cmd.exe
```

Chiudere quindi Miniconda Prompt.

Chiudere anche eventuali finestre di Windows Terminal già aperte e riaprire **Windows Terminal** con il profilo:

Prompt dei comandi

Verificare che Conda sia disponibile:

```
conda --version
```

Ad esempio:

```
conda 26.x.x
```

`conda init cmd.exe` deve normalmente essere eseguito una sola volta.

4. Creare un ambiente Conda dedicato al corso

È buona pratica **non lavorare direttamente nell'ambiente base**.

Creiamo un ambiente separato per il corso, al quale diamo il **generico nome di myenv**:

```
conda create -n myenv python=3.11
```

Dove:

- `conda create` crea un nuovo ambiente;

- `-n myenv` assegna il nome `myenv`;
- `python=3.11` installa Python 3.11 nell'ambiente.

Quando Conda chiede:

```
Proceed ([y]/n)?
```

digitare:

```
y
```

e premere **Invio**.

Per vedere gli ambienti disponibili:

```
conda env list
```

oppure:

```
conda info --envs
```

Un comando **migliore** di creazione dell'ambiente conda, tuttavia, è il seguente:

```
conda create -n myenv -c conda-forge --override-channels python=3.11
```

Con `--override-channels` conda usa solo `conda-forge` e ignora `defaults`. Non mescola quindi pacchetti compilati in modo diverso, che è la causa tipica dei conflitti. Il comando dà lo stesso risultato su qualsiasi PC, qualunque sia la sua configurazione, e non tocca il canale di Anaconda, soggetto a condizioni di licenza per l'uso in azienda.

👉 Dopo questo comando, l'output di:

```
conda list
```

nell'ultima colonna "channel" deve sempre riportare **SOLO** `conda-forge`.

🚫 Se l'ambiente `myenv` esiste già e lo si vuole ricreare occorre prima **cancellarlo** con il comando `conda:`

```
conda remove --name myenv --all -y
```

e poi per sicurezza verificare che la cartella `%USERPROFILE%\miniconda3\envs\myenv` non esista più. Altrimenti la si cancella (dalla cartella padre) con il comando da terminale `cmd`:

```
rmdir /s /q myenv
```

5. Attivare l'ambiente del corso

Ogni volta che si inizia a lavorare al corso, aprire **Windows Terminal** con il profilo **Prompt dei comandi** e digitare:

```
conda activate myenv
```

Il prompt dovrebbe cambiare e mostrare il nome dell'ambiente:

```
(myenv) C:\Users\NomeUtente>
```

La presenza di:

```
(myenv)
```

indica che l'ambiente è attivo.

Una ulteriore verifica dell'ambiente attivo, che fornisce anche il cammino dell'ambiente, è la seguente:

```
echo %CONDA_PREFIX%
```

Il prompt infatti mostra solo il nome dell'ambiente, la variabile dice anche **dove si trova l'ambiente sul disco**.

6. Isolare l'ambiente con PYTHONNOUSERSITE

In alcuni computer Python può trovare package installati in precedenza nella cartella personale dell'utente (il cosiddetto *user-site*), anche quando è attivo un ambiente Conda.

Per evitare che Python utilizzi questi package esterni all'ambiente del corso, nel **Prompt dei comandi (cmd)** si può impostare (**nell'ambiente *myenv* attivo**):

```
set PYTHONNOUSERSITE=1
```

Questa variabile d'ambiente dice a Python:

non aggiungere al percorso di ricerca dei moduli la cartella *user site-packages* dell'utente.

Infatti, essa significa:

```
PYTHON    → riguarda il comportamento di Python
NO        → esclude
USER SITE → la cartella dei package installati a livello utente
=1        → attiva questa opzione
```

In pratica, aiuta a **fare in modo che Python utilizzi soltanto i package disponibili nell'ambiente Conda attivo e nelle sue dipendenze.**

Schema semplificato:

Senza PYTHONNOUSERSITE=1

Ambiente Conda

```
├── package dell'ambiente
└── possibili package installati nella cartella utente (user-site)
    ↑
    possibile interferenza
```

Con:

```
set PYTHONNOUSERSITE=1
```

si ottiene:

Ambiente Conda

```
├── package dell'ambiente
└── ambiente più isolato
```

Questo può essere utile soprattutto durante un corso, perché riduce il rischio che due partecipanti ottengano comportamenti diversi a causa di package Python installati in precedenza sul proprio computer.

Quando eseguire il comando

Una possibile sequenza di avvio della sessione è:

```
conda activate myenv  
set PYTHONNOUSERSITE=1
```

Più avanti, **non ora**, lanceremo VS Code da terminale con:

```
code
```

In questo modo VS Code verrà lanciato con la variabile *PYTHONNOUSERSITE* già impostata e la erediterà.

Attenzione: **set** vale solo per la sessione corrente

Con il Prompt dei comandi, il comando:

```
set PYTHONNOUSERSITE=1
```

imposta la variabile soltanto nella finestra di terminale corrente e nei programmi avviati da essa.

Chiudendo quella finestra di Windows Terminal, l'impostazione viene persa.

Quindi, alla sessione successiva, occorre eventualmente eseguire di nuovo:

```
set PYTHONNOUSERSITE=1
```

Questo comportamento è spesso preferibile durante un corso, perché non modifica permanentemente la configurazione di Windows.

Verificare il valore

Per controllare che la variabile sia impostata:

```
echo %PYTHONNOUSERSITE%
```

L'output dovrebbe essere:

```
1
```

Rendere persistente il set

Per evitare di doversi ricordare di settare la variabile `PYTHONNOUSERSITE` ad ogni nuova istanza di terminale, si può rendere il set persistente con:

```
setx PYTHONNOUSERSITE 1
```

Da sapere:

- `setx` vale solo per **i terminali aperti dopo il comando, non per la finestra corrente**. Lì serve ancora una volta `set PYTHONNOUSERSITE=1`, oppure si chiude e si riapre.
- **i programmi già aperti non la vedono. VS Code va chiuso del tutto**, in tutte le finestre, e riaperto. Da quel momento la ereditano anche i kernel dei notebook.
- non servono i diritti di amministratore. La variabile viene scritta nel registro alla chiave `HKCU\Environment` e vale solo per il tuo utente. Con `/M` in fondo, da un prompt aperto come amministratore, varrebbe per tutti gli utenti del PC.
- riguarda tutti i Python del tuo utente, in ogni ambiente e in ogni terminale (cmd, PowerShell, VS Code). I pacchetti installati con `pip install --user` diventano invisibili ovunque.

Verifica, in un terminale nuovo:

```
echo %PYTHONNOUSERSITE%  
python -m site
```

Il primo comando **deve stampare 1**. In fondo all'output del secondo **deve comparire `ENABLE_USER_SITE: False`**.

Inoltre, un controllo complementare è il seguente:

```
python -m pip list --user
```

Il comando elenca **che cosa c'è nella cartella utente** e `python -m site --user-site` ne stampa **il percorso**. Va eseguito con la variabile non impostata, altrimenti l'elenco risulta vuoto. Se lì non c'è nulla che ti serva, svuotare quella cartella elimina il problema alla radice. La variabile resta allora come protezione per il futuro.

Rimuovere l'impostazione nella sessione corrente

Per cancellare la variabile:

```
set PYTHONNOUSERSITE=
```

Dopo questo comando:

```
echo %PYTHONNOUSERSITE%
```

non dovrebbe più mostrare **1**.

7. Verificare Python

Con l'ambiente attivo:

```
python --version
```

Ad esempio:

```
Python 3.11.x
```

Per verificare quale eseguibile Python viene utilizzato:

```
where python
```

Il primo percorso mostrato dovrebbe fare riferimento alla cartella dell'ambiente Conda **myenv**.

Per verificare **esattamente quale interprete Python sta eseguendo il comando**, si può usare anche:

```
python -c "import sys; print(sys.executable)"
```

Il significato è:

- **python** avvia l'interprete Python attualmente attivo;
- **-c** significa **esegui direttamente il codice indicato tra virgolette**;
- **import sys** importa il modulo di sistema **sys**;
- **sys.executable** contiene il percorso completo dell'interprete Python in uso;
- **print(...)** visualizza quel percorso.

Esempio di risultato:

```
C:\Users\NomeUtente\miniconda3\envs\myenv\python.exe
```

Questo comando è molto utile per controllare che sia effettivamente in uso il Python dell'ambiente Conda del corso.

8. Aggiornare `pip`

Con l'ambiente attivo:

```
python -m pip install --upgrade pip
```

Per verificare:

```
python -m pip --version
```

9. Installare i package con Conda

Con **l'ambiente conda del corso attivo** ⚠:

```
conda activate myenv
```

i package si installano con:

```
conda install NOME_PACKAGE
```

Per esempio:

```
conda install numpy
```

Per installare più package insieme:

```
conda install numpy pandas matplotlib scikit-learn
```

Per **installare l'intera suite di package del corso**, dalla cartella **padre**:

```
conda install -n myenv -c conda-forge --override-channels --file  
requirements_TM_conda.txt
```

Per questo specifico corso, c'è anche un package da installare con **pip**, `it_core_news_sm`, dopo il controllo a secco (**--dry-run**):

```
python -m pip install --dry-run -r requirements_TM_pip.txt
```

e poi, se tutto va bene, lo si installa veramente:

```
python -m pip install -r requirements_TM_pip.txt
```

Ovviamente, in `conda list` ora compare un package (`it-core-news-sm`) con **channel=pypi**.

👉 **L'installazione conda / pip è terminata.** Vediamo ora alcuni *topics* dell'installazione più in dettaglio.

Specificare il canale

Un **canale Conda** è il repository dal quale Conda scarica i package. Non è obbligatorio specificarlo nel comando conda di installazione.

Tuttavia, un canale molto usato in ambito scientifico e Data Science è **conda-forge**.

Raccomandazione.

Usare `conda-forge` come unico canale ed escludere il canale `defaults` (tramite l'opzione `--override-channels`); il canale `defaults`, tra l'altro, richiede la [licenza commerciale](#) per aziende con più di 200 dipendenti/collaboratori.

Per il corso adotteremo quindi `conda-forge` come **unica sorgente** dei package, in modo da rendere l'ambiente più coerente e riproducibile ed evitare mescolanze tra package provenienti da canali diversi.

La forma consigliata di installazione conda sarà quindi questa, con l'opzione `-c conda-forge`:

```
conda install -c conda-forge --override-channels NOME_PACKAGE
```

L'opzione `--override-channels` dice appunto a Conda di ignorare i canali già configurati nel sistema, compreso `defaults`, e di usare solo quelli indicati con `-c`.

`-c` è l'abbreviazione di `--channel`

Installare i package necessari al corso

Per esempio:

```
conda install -c conda-forge --override-channels numpy pandas matplotlib scikit-learn
```

Per controllare i package installati:

```
conda list
```

👉 **conda list** riporta anche la sorgente del package.

🔗 Ecco il **significato dell'output di conda list**:

conda list elenca i pacchetti installati in un ambiente conda, cioè una cartella isolata che contiene un interprete Python e un insieme di pacchetti.

Le righe di intestazione.

- **packages in environment at C:\Users\Utente\miniconda3:** dice quale ambiente stiamo guardando.

Le quattro colonne.

Name: il nome del pacchetto. **Version:** la versione del software. **Build:** la specifica "confezione" di quella versione. Della stessa versione possono esistere più build, per versioni diverse di Python o ricompilate dopo una correzione. **Channel:** il canale, cioè il deposito online da cui il pacchetto è stato scaricato:

- se la cella è vuota, il pacchetto viene dai canali predefiniti di Anaconda (*defaults*)
- *conda-forge* è il canale gestito dalla comunità
- *pypi* non è un canale conda: indica un pacchetto installato con *pip* da PyPI (Python Package Index, l'archivio dei pacchetti Python).

Regola pratica: **esplicitare sempre l'ambiente con -n <env> (o -p <percorso>) anche quando è già attivo:** evita la classe di errori «comando finito nell'ambiente sbagliato» dovuti a **distrazione** (l'ambiente conda attivo era stato modificato!!).

pip list

Se a questo punto facciamo:

```
python -m pip list
```

otteniamo comunque una lista di pacchetti! E' un errore? No!

```
python -m pip list
```

mostra tutti i pacchetti Python presenti nell'ambiente, non solo quelli installati da pip. Le librerie installate da conda portano con sé i metadati standard che pip sa leggere, quindi compaiono anche lì.

Per sapere chi ha installato cosa:

```
conda list | findstr pypi
```

elenca solo i pacchetti installati con pip. Se non stampa nulla, pip non ha installato niente.

```
python -m pip list -v
```

aggiunge le colonne *Installer* (**conda** o **pip**) e *Location*, che deve essere sempre la cartella dell'ambiente.

Strategia di installazione

Privilegiare sempre **conda install**; ricorrere a **python -m pip install** solo per i pacchetti non disponibili tramite **Conda**.

Ci sono però **due importanti accortezze**.

1. Rispettare l'ordine temporale

Conviene installare **prima tutto il possibile con Conda** e, solo in un secondo momento, **ciò che resta con pip**.

Il motivo è che Conda, quando risolve l'ambiente, non gestisce in modo completo i pacchetti già installati da pip. Se si installa qualcosa con pip e poi si esegue un successivo:

```
conda install ...
```

Conda può modificare o sostituire file e dipendenze su cui il pacchetto installato con pip faceva affidamento, rendendo incoerente o non funzionante l'installazione precedente.

Per questo motivo è consigliabile lasciare **pip "in coda"**, quindi:

```
1. conda install ...
2. conda install ...
3. conda install ...
4. python -m pip install ... ← solo i package che mancano (non installabili con conda)
```

2. Non alternare Conda e pip sullo stesso pacchetto

Una volta che un pacchetto è stato installato con pip, gli aggiornamenti di **quel pacchetto** devono essere eseguiti **sempre con pip, non con Conda, e viceversa**.

Per esempio, se è un pacchetto stato installato con:

```
python -m pip install nome-pacchetto
```

gli aggiornamenti successivi dovrebbero essere eseguiti con:

```
python -m pip install --upgrade nome-pacchetto
```

e non con:

```
conda update nome-pacchetto
```

👉 Mescolare i due gestori sullo stesso package è una delle cause più comuni di ambienti Python incoerenti.

Forma consigliata per pip

In ogni caso, preferire sempre:

```
python -m pip install NOME_PACKAGE
```

alla forma:

```
pip install NOME_PACKAGE
```

Il significato è:

- **python** richiama l'interprete Python attualmente attivo;
- **-m** significa **esegui il modulo pip del Python attivo** → così è l'interprete Python attivo a eseguire il **PROPRIO** modulo **pip**.
- **pip**, come noto, è un modulo che gestisce l'installazione dei package;
- **install NOME_PACKAGE** chiede a pip di installare il package indicato.

Cioè, in sintesi, **python -m pip install pandas** significa: usa **questo Python attivo** per eseguire **il suo pip** e installare **pandas**.

Questa forma è preferibile al semplice e più veloce **pip install** perché riduce il rischio di usare accidentalmente un **pip** appartenente a un'altra installazione di Python.

È quindi particolarmente utile quando sul PC sono presenti:

- più versioni di Python;
- più ambienti Conda;
- Python installato anche fuori da Miniconda.

Comandi `pip` da evitare

🚫 Evitare sempre, come detto:

```
pip install NOME_PACKAGE
```

Il problema è semplice: **quale `pip` viene eseguito?**

Se sul PC sono presenti più versioni di Python o più ambienti, il comando `pip` trovato nel `PATH` potrebbe non essere quello associato all'interprete Python dell'ambiente attivo.

🚫 Evitare (quasi sempre):

```
python -m pip install --user NOME_PACKAGE
```

L'opzione `--user` installa il package nello **user-site**, cioè in una cartella personale dell'utente Windows, **non nell'ambiente Conda del corso**.

Su Windows il percorso dello *user-site* è tipicamente simile a:

```
%APPDATA%\Python\PythonXX\site-packages
```

dove: `%APPDATA%` = `C:\Users\User\AppData\Roaming`.

Sul mio PC lo *user-site* è qui:

Quindi:

```
ambiente myenv
├── package dell'ambiente
└── NON contiene i package installati con --user

user-site di Windows
└── package installati con --user
```

Con `venv` (il gestore di ambienti virtuali nativo di Python, alternativo a conda), il comando `pip install --user` viene generalmente rifiutato perché lo *user-site* non è visibile nell'ambiente. Con un ambiente Conda, invece, questo comando **può installare il package fuori dall'ambiente senza che l'errore sia evidente**: si può quindi credere di avere installato il package nel proprio ambiente quando in realtà si trova nello *user-site*.

L'opzione `--user` di pip (quando si deve usare pip) è quindi **lo strumento sbagliato**, perché non isola l'ambiente.

Il meccanismo dello *user-site* e il modo per neutralizzarne l'uso sono spiegati nella sezione dedicata a: `set PYTHONNOUSERSITE=1`.

Perché abbiamo prima detto “quasi sempre” e non “sempre”?

`--user` ha alcuni usi legittimi fuori dal perimetro di questa guida. Il caso classico è l'uso del **Python di sistema senza permessi di amministratore**, quando si vuole installare un package soltanto per l'utente corrente.

Un altro caso riguarda alcuni tool a riga di comando che si desidera rendere disponibili all'utente indipendentemente dall'ambiente; per questo scenario oggi è spesso preferibile usare strumenti dedicati come `pipx`.

Nel nostro corso, che utilizza sempre un ambiente Conda dedicato, la **regola pratica del pip** è così riassumibile:

```
NON installare package con pip (se non per i package NON installabili con conda,
ed alla fine)
se necessario installare con pip, NON usare 'pip install ...'
usare, solo quando necessario: 'python -m pip install ...'
comunque, in questi casi, NON usare 'python -m pip install --user ...'
in seguito, se si era dovuto installare un package con pip, fare il suo eventuale
upgrade solo con pip stesso (e mai con conda)
```

Riavvio del kernel

Dopo aver installato un package da terminale, se VS Code o altra IDE erano già aperti, di norma è necessario riavviare il kernel di VS Code (o altra IDE) affinché il modulo appena installato venga **riconosciuto correttamente** dal codice in esecuzione.

Se non si riavvia il kernel, il tentativo di importare il pacchetto (es. `import nome_pacchetto`) produrrà quasi sicuramente un errore di tipo `ModuleNotFoundError`, poiché il processo Python attualmente attivo ha già caricato in memoria l'elenco dei moduli disponibili al momento del suo avvio.

💡 L'alternativa per non riavviare il kernel: **i comandi "magici"**

Se vsi vuole evitare di riavviare il kernel e perdere le variabili caricate in memoria, si può installare il pacchetto direttamente all'interno di una cella del notebook usando i "comandi magici" di Jupyter. In questo modo il kernel si aggiorna automaticamente senza bisogno di riavvii:

Per **pip**:

- `%pip install <nome_pacchetto>`

Per **conda**:

- `%conda install <nome_pacchetto>`

🔧 Come **riavviare rapidamente il kernel in VS Code**

Se si è già installato il pacchetto dal terminale esterno o integrato (a VS Code), si può riavviare il kernel in due modi rapidi:

1. Cliccare sul pulsante *Restart* (l'icona a forma di freccia circolare) nella barra degli strumenti in alto all'interno del notebook di VS Code.
2. In alternativa, premere *Ctrl + Shift + P* (o *Cmd + Shift + P su Mac*), digitare "Jupyter: Restart Kernel" e premere *Invio*.

Al contrario, se Visual Studio Code non era aperto al momento della installazione del package da terminale, è sufficiente aprire VSC e poi avviare il kernel del notebook: il nuovo ambiente **viene caricato da zero**. In questo caso, Python rileva immediatamente tutti i moduli aggiornati e si possono importare subito in VSC senza dover compiere alcuna operazione di riavvio.

Installazioni da IDE??

🚫 **Importante: non installare package dalle celle della IDE con `!pip` o `!conda`**

Evitare di eseguire nelle celle della IDE comandi come:

```
!pip install nome_pacchetto
```

oppure:

```
!conda install nome_pacchetto
```

Il prefisso **!** fa eseguire il comando attraverso una **shell separata**. In quel contesto non è garantito che il `pip` o il `conda` individuato corrisponda esattamente all'ambiente Python associato al **kernel** del notebook.

Il rischio è quindi di installare il package in un ambiente diverso da quello effettivamente usato dal notebook, creando incoerenze o problemi nel kernel.

Regola consigliata per il corso: installare sempre i package da **Windows Terminal**, dopo avere attivato l'ambiente Conda, oppure, se il package non è disponibile con conda, con il `pip`.

10. Interprete Python e kernel Jupyter

Poiché durante il corso lavoreremo esclusivamente con **notebook Jupyter (.ipynb)**, è importante distinguere due concetti: **interprete Python** e **kernel Jupyter**.

L'interprete Python

L'**interprete** è il programma che esegue il linguaggio Python.

Nel nostro ambiente Conda, `myenv`, l'interprete è il file `python.exe` installato dentro quell'ambiente. In forma semplificata:

```
Ambiente Conda: myenv
├── python.exe
│   ↑
│   interprete Python
```

Quando creiamo l'ambiente con:

```
conda create -n myenv python=3.12
```

stiamo creando un ambiente virtuale (conda) che contiene, tra le altre cose, uno specifico **interprete Python 3.12**.

L'interprete stabilisce:

quale Python stiamo utilizzando, quale pip, e quali package appartengono a quell'ambiente.

👉 Per **eseguire un file python** (suffisso `.py`) in VS Code è necessario scegliere un'interprete (e non un kernel), in questo modo:

```
ctrl + shift + P
↓
Python Select Interpreter
↓
myenv (X.YY.ZZ)
```

Il kernel Jupyter

Un notebook Jupyter non esegue direttamente le celle semplicemente "attraverso Python": utilizza un **kernel**.

Il **kernel** è un processo attivo che:

- usa un determinato interprete Python;
- riceve il codice delle celle dal notebook;
- lo esegue;
- restituisce i risultati a VS Code;
- **mantiene in memoria lo stato della sessione.**

Per esempio, se in una cella eseguiamo:

```
x = 100
```

e successivamente, in un'altra cella:

```
print(x)
```

otteniamo:

```
100
```

perché il **kernel è rimasto attivo e conserva `x` in memoria**.

Se eseguiamo:

```
Restart Kernel
```

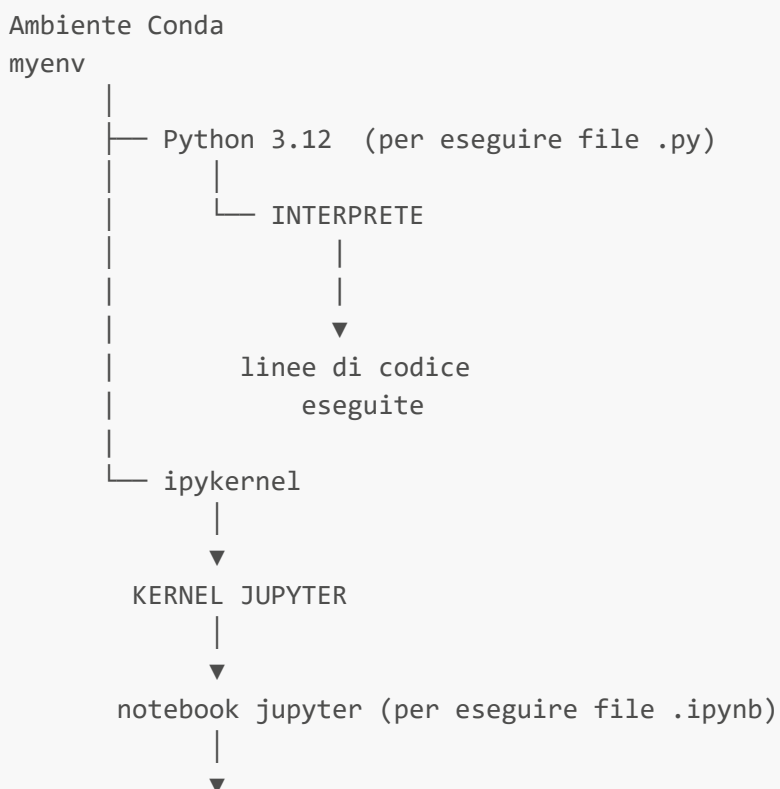
la **memoria del kernel viene azzerata e le variabili create in precedenza non esistono più**.

La differenza essenziale è quindi:

INTERPRETE = quale installazione di Python viene utilizzata

KERNEL = la sessione attiva che usa quell'interprete
per eseguire le celle del notebook

👉 La relazione completa è dunque:



celle di codice
eseguite

Installare `ipykernel`

Perché l'ambiente Conda possa essere utilizzato come kernel di un notebook Jupyter, **installiamo il package `ipykernel`**

Prima, come sempre, dobbiamo attivare l'ambiente **da terminale**:

```
conda activate myenv
```

e quindi installare:

```
conda install ipykernel
```

Ancor meglio è forzare esplicitamente l'uso del canale *conda-forge*:

```
conda install -c conda-forge --override-channels ipykernel
```

Come detto, dunque, `ipykernel` è il **componente che collega l'interprete Python dell'ambiente alla modalità di esecuzione richiesta dai notebook Jupyter**.

In termini semplici:

VS Code / Jupyter (interfaccia: celle, output)
| ▲
| | protocollo Jupyter (messaggi: "esegui questo codice", "ecco il
risultato")
▼ |
┌───────────┴───────────┐
| kernel = processo python.exe
| che esegue ipykernel
| (Python + pacchetti dell'ambiente) |
└───────────┴───────────┘

👉 Con altri strumenti, come **Jupyter Notebook o JupyterLab**, se essi vengono avviati da un ambiente diverso da `myenv`, è necessario anche **registrare il kernel**, cioè indicare a Jupyter quale interprete Python avviare. [Diversamente, se l'ambiente di avvio è `myenv`, il kernel predefinito è già quello dell'ambiente, e la registrazione non serve].

La sequenza completa e corretta per **installare e registrare il kernel** è dunque:

```
conda activate myenv
conda install -c conda-forge --override-channels ipykernel
python -m ipykernel install --user --name myenv --display-name "Python
(myenv)"
```

👉 In VS Code, invece, la registrazione (terzo comando) non è necessaria: le estensioni **Python** e **Jupyter** rilevano gli ambienti Conda disponibili e permettono di selezionarli direttamente. Resta necessaria l'installazione di **ipykernel** (secondo comando); se manca, VS Code propone di installarlo al primo avvio di una cella. In questo caso, comunque, conviene sempre installarlo da terminale, anziché delegare l'operazione a VS Code: in questo modo scegliamo noi il canale da cui scaricarlo (*conda-forge*) e, se qualcosa va storto, **vediamo per intero i messaggi di errore**.

11. Avviare Visual Studio Code dall'ambiente Conda

Questo è il flusso consigliato.

Prima attivare l'ambiente:

```
conda activate myenv
```

Poi lanciare Visual Studio Code:

```
code
```

Oppure, se ci si trova già nella cartella del progetto:

```
code .
```

Il punto `.` significa:

apri in VS Code la cartella corrente.

Esempio:

```
cd C:\CorsoPython
conda activate myenv
code .
```

In questo modo VS Code viene aperto **direttamente sulla cartella corrente (grazie a code .)**, quella del progetto, ed **eredita le variabili di ambiente Windows dell'ambiente attivo**, cioè le variabili di ambiente Windows (in primis, Path) al momento del lancio (**l'eredità anche senza .**). È il meccanismo normale in Windows per cui un processo *figlio*, cioè avviato da un altro detto *padre*, parte con le variabili del padre.

Note:

- l'ereditarietà delle variabili di ambiente Windows dal prompt **vale solo se VS Code non è già in esecuzione**. Se un'istanza (cioè una copia di VS Code in esecuzione) è già aperta, **code** non avvia un nuovo VS Code: passa la richiesta a quell'istanza, che apre una nuova finestra. **Il padre della nuova finestra è quindi il VS Code già aperto, non il nostro prompt!**. La finestra riceve perciò le variabili di quell'istanza e non quelle appena modificate da **conda activate**. Se l'istanza era stata avviata dal menu *Start*, nel suo *Path* le cartelle di *myenv* non ci sono. È un **tranello frequente**, che spiega molti casi di "ho attivato l'env ma VS Code non lo vede". La regola pratica è dunque semplice: **chiudiamo tutte le finestre di VS Code prima di lanciare code**.
- 🖱️ in ogni caso, anche quando le variabili sono ereditate correttamente, questa modalità (di attivazione di VS Code con **code** da terminale) non garantisce automaticamente che il notebook *.ipynb* userà *myenv* come kernel (vedi i passi successivi): tuttavia, riduce le ambiguità iniziali e rende più prevedibile il rilevamento degli ambienti Python.
- ⚠️ dopo l'apertura, sarà comunque **necessario verificare il kernel selezionato nel notebook**, in alto a destra.

🔴 Esiste una **scorciatoia di navigazione nell'albero della cartelle di Windows**: navigare da Explorer è molto più veloce che da terminale!

E' sufficiente infatti, da terminale, eseguire (una volta sola, in modo persistente per le successive sessioni di terminale):

```
conda init cmd.exe
# oppure
conda init powershell
```

Se si ottiene "No action taken" significa che l'inizializzazione era già stata fatta.

A questo punto dobbiamo chiudere e riaprire il terminale, **conda init** infatti modifica impostazioni che il terminale legge solo all'avvio.

Si aprirà un terminale senza il prefisso base (tipico di conda), ma sarà sufficiente eseguire:

```
conda --version
conda activate <myenv>
```

🖱️ A questo punto il terminale è **equivalente ad un miniconda prompt**.

A questo punto la scorciatoia è attiva e possiamo dunque utilizzarla, in questo modo:

- navigare nell'explorer di Windows sino alla cartella desiderata (es. *myenv*) e selezionare con il mouse il percorso in alto nella barra
- scrivere sulla barra *cmd* o *powershell*
- premere invio e si aprirà un terminale (del tipo scelto) **impostato su quel percorso** (senza doverci navigare dal terminale!!).

12. Installare le estensioni Python e Jupyter in VS Code

⚠ Si installano **una volta sola per ogni installazione di VS Code**. La creazione o la cancellazione degli ambienti conda non le tocca.

⚠ VS CODE può **proporre automaticamente le estensioni necessarie Python e Jupyter** quando si apre per la prima volta un file *.ipynb*; tuttavia, per ottenere una configurazione stabile e prevedibile, è consigliabile installarle esplicitamente.

In Visual Studio Code aprire **Extensions** dalla barra laterale sinistra oppure usare:

```
Ctrl + Shift + X
```

Installare le seguenti estensioni, tutte pubblicate da **Microsoft**.

1. Pylance

Fornisce supporto avanzato alla scrittura del codice Python:

- completamento automatico;
- analisi statica;
- suggerimenti sui tipi;
- navigazione nel codice.

2. Python

È l'estensione principale per l'integrazione tra VS Code e Python.

Serve, tra le altre cose, per:

- individuare gli interpreti Python;
- lavorare con gli ambienti Conda;
- integrare gli strumenti Python in VS Code.

3. Python Debugger

Aggiunge il supporto al debugging Python:

- breakpoint;
- esecuzione passo-passo;
- ispezione delle variabili;
- analisi dello stack delle chiamate.

4. Python Environments

Facilita la gestione degli ambienti Python e Conda direttamente da VS Code.

Nel nostro caso l'ambiente da utilizzare sarà:

```
myenv
```

5. Jupyter

Poiché durante il corso lavoreremo **solo con notebook** `.ipynb`, questa estensione è indispensabile.

Permette di:

- aprire e modificare notebook Jupyter;
- eseguire le celle;
- scegliere il kernel;
- visualizzare output, grafici e tabelle;
- gestire la sessione del notebook.

Stampare un file Markdown aperto in VS Code

VS Code permette di visualizzare direttamente la **preview renderizzata** di un file Markdown `.md`.

Per aprirla:

```
Ctrl + Shift + V
```

oppure:

```
Ctrl + Shift + P  
→ Markdown: Open Preview
```

Per la sola visualizzazione non serve installare alcuna estensione aggiuntiva.

Per **stampare** un file Markdown, invece, il flusso più pratico è:

```
file .md  
↓  
Markdown Preview  
↓  
esportazione in PDF  
↓  
stampa del PDF
```

Se si desidera fare tutto da VS Code, è possibile installare un'estensione dedicata, ad esempio:

Markdown PDF

che aggiunge comandi di esportazione, ad esempio:

Markdown PDF: Export (pdf)

Una volta creato il PDF, lo si può aprire e stampare normalmente.

In sintesi

- per leggere il Markdown: **Ctrl + Shift + V**
- per stamparlo: esportarlo prima in PDF e poi stampare il PDF

Le estensioni da installare

Pylance
Python
Python Debugger
Python Environments
Jupyter

Tutte devono avere come publisher:

Microsoft

L'estensione **Markdown PDF** è opzionale: serve soltanto se si desidera esportare e stampare comodamente i file Markdown direttamente da VS Code.

13. Selezionare il kernel in VS Code (con file .ipynb)

Aprire il notebook **.ipynb**.

In alto a destra in genere compare:

Select Kernel

ed occorre allora fare clic su di esso.

👉 Non sempre compare **Select Kernel**: a volte VS Code seleziona il kernel automaticamente, leggendo i **metadati** del notebook (le informazioni salvate nel file `.ipynb` insieme alle celle, tra cui il **nome del kernel** usato l'ultima volta) oppure ricordando la scelta fatta in precedenza. In questo caso, in alto a destra **compare direttamente il nome del kernel**.

Prima di iniziare ad eseguire le celle del notebook conviene comunque **controllare quel nome**: se il notebook proviene da un altro computer, i metadati si riferiscono all'ambiente di chi l'ha creato, e VS Code potrebbe dunque proporre un ambiente diverso da quello che vogliamo usare. Per cambiarlo basta fare clic sul nome e selezionare `myenv`.

Tornando al caso più frequente, dopo il click su **Select kernel**, a seconda dei casi VS Code mostra **uno dei due elenchi seguenti**.

Caso 1 – VS Code suggerisce già un ambiente. In cima all'elenco compare **il kernel proposto**, ad esempio:

```
myenv (Python 3.11.11)  ~\miniconda3\envs\myenv\python.exe
Select Another Kernel...
```

Se nome e percorso corrispondono all'ambiente che vogliamo usare, basta selezionare la prima riga. Altrimenti scegliere **Select Another Kernel...**, compare una lista e ci si allaccia al caso 2.

Caso 2 – VS Code non ha suggerimenti.

VS Code mostra una finestra con diverse sorgenti possibili del kernel:

```
Python Environments...
Jupyter Kernel...
Existing Jupyter Server...
```

Nel nostro caso, con **Miniconda + ambiente Conda locale**, la scelta corretta è:

```
Python Environments...
```

Questa voce **mostra gli ambienti Python disponibili sul computer, compresi gli ambienti Conda**.

Selezionare quindi l'ambiente del corso, ad esempio:

```
Python 3.12 (myenv)
```

Il flusso generale è dunque:

```
Cosa facciamo noi (in VS Code)

Select Kernel
↓
```

Python Environments...



myenv

Cosa fa VS Code (dietro le quinte)

all'esecuzione della prima cella avvia il kernel:

python.exe dell'ambiente myenv + ipykernel



per ogni cella eseguita:

invia il codice al kernel ⇔ riceve output, risultati ed errori



mostra i risultati sotto la cella



In ogni caso, in qualsiasi dei due casi, prima di eseguire le celle del notebook conviene verificare il **percorso** mostrato accanto al nome del kernel: è il percorso del **python.exe** dell'ambiente (~ indica la cartella dell'utente) e indica senza ambiguità quale ambiente verrà usato.

Significato delle tre opzioni

Vediamo ora più in dettaglio il significato delle tre opzioni del caso 2.

Python Environments...

Permette di scegliere uno degli ambienti Python installati sul computer:

- ambienti Conda;
- ambienti **venv**;
- altre installazioni Python locali.

È l'opzione da utilizzare nel corso.



In VS Code, quando si sceglie *Python Environments...*, **il kernel viene costruito a partire da uno specifico interprete Python appartenente a un ambiente Conda o virtuale**.

Jupyter Kernel...

Permette di scegliere un **kernel Jupyter già registrato** nel sistema tramite una *kernelspec*.

Per esempio, un kernel può essere stato registrato manualmente con:

```
python -m ipykernel install --user --name myenv
```

Nel nostro approccio questo passaggio **non è normalmente necessario**, perché VS Code può usare direttamente l'ambiente Conda tramite:

Python Environments...

Existing Jupyter Server...

Permette di collegare VS Code a un **Jupyter Server già esistente**, locale o remoto.

È utile, ad esempio, quando il notebook viene eseguito su:

- server aziendali;
- macchine remote;
- infrastrutture GPU;
- JupyterHub;
- server di laboratorio.

Nel nostro scenario locale su Windows **non è necessario**.

In sintesi:

Per il corso:

Select Kernel



Python Environments...



myenv

A quel punto il notebook utilizzerà:

myenv



├─ interprete Python 3.12

├─ ipykernel

└─ package installati nell'ambiente

🚀 Tutte le celle del notebook saranno quindi eseguite dal **kernel associato all'ambiente myenv**.

14. Flusso di lavoro consigliato durante il corso

Ogni volta che si inizia una sessione:

1. Aprire Windows Terminal



2. Selezionare Prompt dei comandi



3. Attivare l'ambiente Conda
↓
4. Impostare PYTHONNOUSERSITE
↓
5. Spostarsi nella cartella del progetto
↓
6. Avviare VS Code
↓
7. Aprire il notebook e selezionare il kernel myenv

Comandi:

```
conda activate myenv
set PYTHONNOUSERSITE=1
cd C:\CorsoPython
code .
```

Schema:

```
Windows Terminal
|
▼
Prompt dei comandi (cmd)
|
|   conda activate myenv
|   ▼
(myenv)
|
|   cd C:\CorsoPython
|   ▼
Cartella del progetto
|
|   code .
|   ▼
Visual Studio Code
|
▼
Notebook .ipynb
|
▼
Kernel: myenv
```

15. Terminare la sessione

Quando si è terminato di lavorare, è possibile disattivare l'ambiente con:

```
conda deactivate
```

Il prefisso:

```
(myenv)
```

scomparirà dal prompt.

Problemi comuni

conda non viene riconosciuto

Se compare un messaggio simile a:

```
'conda' non è riconosciuto come comando interno o esterno...
```

aprire **Miniconda Prompt** dal menu Start ed eseguire:

```
conda init cmd.exe
```

Chiudere completamente Windows Terminal e riaprirlo. [conda init infatti modifica impostazioni che il terminale legge solo all'avvio.]

code non viene riconosciuto

Se compare:

```
'code' non è riconosciuto come comando interno o esterno...
```

chiudere e riaprire Windows Terminal dopo l'installazione di VS Code.

Se il problema rimane, verificare l'installazione di Visual Studio Code e che il comando `code` sia disponibile nel PATH.

VS Code utilizza il Python sbagliato

Premere:

```
Ctrl + Shift + P
```

quindi:

```
Python: Select Interpreter
```

e scegliere:

```
myenv
```

Il terminale integrato di VS Code apre PowerShell invece di cmd

In VS Code è possibile selezionare il profilo del terminale desiderato.

Aprire il terminale integrato con:

```
Ctrl + `
```

Dal menu del terminale scegliere:

```
Select Default Profile
```

e quindi:

```
Command Prompt
```

Aprire infine un nuovo terminale.

A questo punto il prompt dovrebbe avere una forma simile a:

```
C:\CorsoPython>
```

e, dopo l'attivazione dell'ambiente:

```
(myenv) C:\CorsoPython>
```

In caso di problemi con VS Code / Jupyter

Se VS Code non riconosce correttamente l'ambiente Conda, il kernel non compare oppure il notebook non viene eseguito come previsto, aprire la **Command Palette** con:

```
Ctrl + Shift + P
```

e provare, nell'ordine, i seguenti comandi.

1. Developer: Reload Window

Ricarica la finestra di VS Code e riavvia l'interfaccia e le estensioni senza chiudere il progetto.

È il primo tentativo da fare quando VS Code sembra non aggiornare correttamente lo stato degli ambienti o delle estensioni.

2. Python Environments: Refresh All Environment Managers

Forza VS Code a rieseguire la ricerca degli ambienti Python disponibili.

È particolarmente utile quando:

- è stato appena creato un nuovo ambiente Conda;
- l'ambiente `myenv` non compare;
- VS Code mostra un elenco di ambienti non aggiornato.

3. Python: Select Interpreter

Permette di scegliere esplicitamente l'interprete Python dell'ambiente Conda.

Nel nostro caso selezionare:

```
myenv
```

oppure una voce simile a:

```
Python 3.12 (myenv)
```

4. Notebook: Select Notebook Kernel

Permette di scegliere nuovamente il kernel utilizzato dal notebook.

Selezionare:

```
Python Environments...
```

e quindi:

```
myenv
```

Sequenza consigliata di troubleshooting

```
Ctrl + Shift + P
    ↓
Developer: Reload Window
    ↓
Python Environments: Refresh All Environment Managers
    ↓
Python: Select Interpreter
    ↓
myenv
    ↓
Notebook: Select Notebook Kernel
    ↓
Python Environments...
    ↓
myenv
```

Se il problema persiste, verificare anche che nell'ambiente sia installato:

```
conda install ipykernel
```

e che in VS Code siano installate e abilitate le estensioni Microsoft:

```
Python
Pylance
Python Debugger
Python Environments
Jupyter
```

Riepilogo dei comandi

```
:: inizializzazione: normalmente una sola volta (persistente per tutte le sessioni successive di terminale cmd)
conda init cmd.exe

:: creare l'ambiente del corso
conda create -n myenv python=3.12

:: attivarlo
conda activate myenv

:: isolare Python dai package installati a livello utente
set PYTHONNOUSERSITE=1

:: verificare Python
python --version
where python
python -c "import sys; print(sys.executable)"

:: installare i package del corso
conda install -c conda-forge --override-channels numpy pandas matplotlib scikit-learn

:: installare il kernel Jupyter
conda install ipykernel

:: entrare nella cartella del corso
cd C:\CorsoPython

:: avviare VS Code dalla cartella corrente
code .

:: al termine
conda deactivate
```

Concetto fondamentale

Un **ambiente Conda** è un'installazione Python isolata.

Possiamo quindi avere sullo stesso computer ambienti diversi:

```
base
├── myenv           → Python 3.12 + librerie del corso
├── progetto-A     → altre librerie
└── progetto-B     → altra versione di Python
```

Questo riduce i conflitti tra versioni di Python e librerie e rende l'ambiente del corso più semplice da riprodurre e gestire.

